

Prototipo de Arquitectura orientada a servicios en la Nube: Plataforma "AnalytiCore"

1. Contexto del Negocio

Una startup de análisis de datos, "AnalytiCore", desea ofrecer un servicio en línea que permita a los usuarios enviar textos para un análisis simple de sentimiento y extracción de palabras clave. El objetivo es construir un prototipo funcional desplegado en la nube que demuestre la viabilidad técnica de una arquitectura políglota (usando diferentes lenguajes de programación) y sirva como base para futuras expansiones.

2. Visión General de la Arquitectura

Se requiere una arquitectura orientada a servicios desplegada en la plataforma Render. La arquitectura consta de tres componentes principales, cada uno empaquetado en su propio contenedor Docker para simular un entorno de producción:

3. Requisitos Técnicos y Funcionales por Componente

1. Frontend Web (React): Una aplicación de una sola página que sirve como interfaz de usuario. Se servirá a través de un servidor web ligero como Nginx.
2. Servicio de Submisión (Python): Un servicio web responsable de recibir las solicitudes del frontend, validarlas, persistir el trabajo inicial en la base de datos y orquestar el inicio del análisis.
3. Servicio de Análisis (Java): Un servicio web "worker" que realiza el análisis del texto.

4. Flujo de Datos:

1. Usuario -> Frontend (React): El usuario introduce el texto y lo envía.
2. Frontend -> (API REST) -> Servicio de Submisión (Python): El frontend envía el texto. El servicio Python crea un registro en la base de datos con estado "PENDIENTE" y llama de forma síncrona al servicio Java para iniciar el análisis. Luego, devuelve un 'jobId' al frontend.

3. Servicio de Submisión (Python) -> (API REST Interna) -> Servicio de Análisis (Java): El servicio Python notifica al servicio Java que un nuevo trabajo está listo para ser procesado.
4. Servicio de Análisis (Java) <-> Base de Datos (PostgreSQL): El servicio Java actualiza el estado del trabajo a "PROCESANDO", realiza el análisis, y al finalizar, guarda los resultados y cambia el estado a "COMPLETADO".
5. Frontend -> (API REST) -> Servicio de Submisión (Python): El frontend consulta periódicamente usando el `jobId` para conocer el estado y obtener los resultados una vez que estén listos.

5. Patrones de Arquitectura Cloud Obligatorios

1. Empaquetar las Aplicaciones: Cada componente (Frontend, Python, Java) debe estar en su propia imagen Docker, configurado para un entorno de producción.
2. Exponer Componentes como APIs: La comunicación entre los componentes debe ser exclusivamente a través de APIs RESTful.
3. Servicios sin Estado (Stateless): Ningún servicio debe guardar estado en memoria. Todo el estado de la aplicación (datos de los trabajos) debe externalizarse a la base de datos PostgreSQL gestionada por Render.
4. Configuración externa de la base de datos PostgreSQL
5. Todos los componentes deben seguir una arquitectura limpia.

6. Entregables:

Se debe entregar un único repositorio en GitHub con la siguiente estructura y contenido:

1. Código Fuente:
 - * Estructura de carpetas clara: `/frontend`, `/python-service`, `/java-service`.
 - * Dockerfile, en la raíz de cada una de estas carpetas.
2. Diagrama de Componentes
3. Diagramas de Capas:
 - * Para cada uno de los 3 componentes, un diagrama que muestre su arquitectura interna (Arquitectura limpia).

4. Informe Ejecutivo:

- * Debe tener una extensión máxima de una página.
- * Explicando:
 - * El problema de negocio que se resuelve.
 - * La solución propuesta y su valor.
 - * Los beneficios de la arquitectura elegida (escalabilidad, mantenibilidad, flexibilidad del equipo al usar diferentes tecnologías) en un lenguaje sencillo.